

매일 아침 기억을 잃는 신입과 일하는 법

에이전트에게 지시하는 사람에서, 시스템으로 지휘하는 리더로

이 동료의 이력서는 화려합니다

압도적인 역량

- 전 세계 모든 라이브러리 문서를 아는 지식
- 초당 수천 줄의 코드를 작성하는 속도
- 지치지 않고 24시간 요청을 처리하는 체력

완벽한 대형 언어 모델 (LLM)

치명적인 치매 & 버릇

- 세션 종료시 어제의 기억을 완전히 상실
- 막히면 되묻지 않고 빈칸을 지어냄 (환각)
- 요청하지 않은 파일까지 과잉 수정
- 실제 테스트 없이 "완료했습니다" 허위 보고

매일 아침 기억을 잃는 유능한 신입

신입이 온 게 아니라, 당신이 PL이 된 겁니다



이전의 나 (Individual Contributor)

내가 직접 생각하고, 내가 직접 구현하고, 내가 검수한다.



에이전트를 붙인 나 (Project Leader)

업무를 위임하고, 맥락을 전달하며, 결과를 확인 및 검증한다.

"갑자기 준비 없이 리더가 되었기에, 지금 일하는 방식의 혼란이 오는 것은 당연합니다."

오늘 이 자리가 끝났을 때 손에 쥐는 것

목표 1. 위임의 정밀화

완료 조건·범위 밖·방법 자율성 분리

모호한 지시를 에이전트가 완수 가능한 한 문장 크기로 설계

목표 3. 워크플로우 이식

기록을 남길 위치 1개(인계/상태 파일) + 검사 1개

성실성에 의존하지 않고 다음 세션 입력이 되도록 세팅

목표 2. 탐색의 태도 전환

"없습니다"를 결론이 아닌 가설로 취급

검색 0건이 부재 증거가 아님을 알고 지도/상태 제공

목표 4. 기록 자산화

개인 메모리 vs 팀 공유 버전 관리 구분

개인 툴 메모리를 넘어 저장소 차원의 맥락 기록 체계 구축

PART 1

위임과 확인 다이얼은 두 개입니다

어떤 리더가 에이전트와 가장 일 잘하는 리더인가?

던져주는 리더 vs 분해해 주는 리더

그냥 던지는 리더

"로그인 기능 알아서 구현해줘"

- 사람 팀에선 시니어에게 통함
- 에이전트는 침묵하며 빈칸을 지어냄
- 예상과 완전히 다른 결과물 출몰

세세하게 다 분해하는 리더

"A파일 12줄에 if문 추가하고 B함수 호출해줘"

- 에이전트의 속도 이점 소멸
- "내가 직접 짜는 게 빠르겠다" 피로감
- 리더가 전체 시스템의 결정적 병목

 정답은 분해의 양이 아니라, "무엇을 고정하고 무엇을 풀어주는가"에 있습니다.

필요할 때 항상 되묻는다고 기대할 수 없습니다

사람 신입 개발자

"팀장님, DB 스키마에 결제 상태 필드가 안 보이는데 기존 Enum을 써야 하나요, 아니면 추가해야 하나요?"

→ 막히면 멈추고 질문을 던진다.

AI 에이전트

"결제 상태 필드가 없으므로 기존 코드를 임의로 수정하여
`STATUS\_UNKNOWN\_V2`를 임의로 추가하고 작업을 마쳤습니다."

→ 되묻지 않고 그럴듯한 빈칸을 채운다.

다이얼은 하나가 아니라 둘입니다

Low 위임 / Low 확인

방임형 / 무관심

아무 시너지도 없음

Low 위임 / High 확인

마이크로매니저

내가 다 지시, 내가 병목

High 위임 / Low 확인

묻지마 던지기

환각 사고 빈번

High 위임 / High 확인 (정답)

시스템적 리더

방법 자율 + 자동 대조

무조건 고정할 3가지

- 1. 완료 조건 (Done Definition)
- 2. 범위 밖 (Out of Scope)
- 3. 작업의 크기 (Single Task Size)

 풀어줄 것: 구현 방법과 알고리즘 선택 (자율성)

이 일을 맡길 준비가 되었습니까?

× 모호하고 위험한 지시

"결제 실패 원인 조사하고 코드 고쳐줘"

✓ 3가지 다이얼이 고정된 지시 카드 (위임 카드)

- 완료 조건: `logs/payment.log`에서 500 에러 원인을 찾고 테스트 케이스 1개 작성 통과
- 범위 밖: 결제 PG사 연동 모듈 로직 본체는 수정 금지
- 허용할 방법: 재시도(Retry) 핸들러 추가 또는 예외 로직 분기 자유

확인을 앞으로 당기면 판단이 대조가 됩니다

기존: 확인을 뒤에 둘 때

사람 리더의 "주관적 판단"

에이전트가 수천 줄 짰 코드를 보고 "잘 짜였나?" 사람이 직접 눈으로 읽고 주관적으로 판단한다.

→ 리더 피로도 극대화

개선: 완료 조건을 사전에 확정할 때

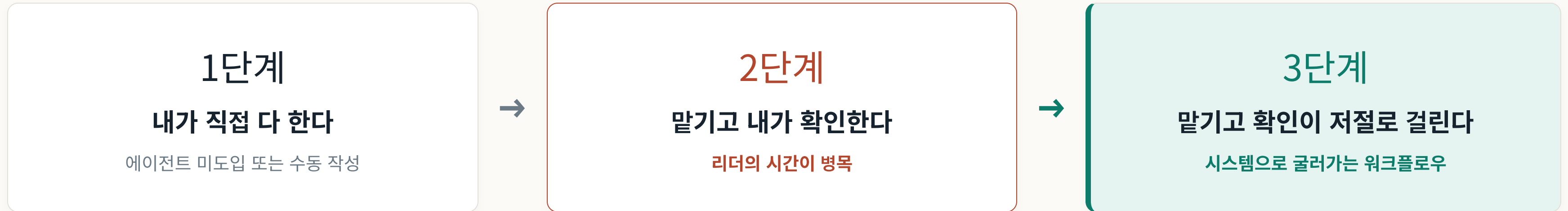
기계에 넘길 수 있는 "대조"

완료 조건을 테스트/스크립트로 명시하고 결과 일치 여부만 대조한다.

→ 대조 작업은 자동화/기계 이관 가능

 업무 크기 기준 = 완료 조건을 한 문장으로 쓸 수 있는 크기

확인을 조이면 내가 병목이 됩니다



"2단계에 머무는 리더는 에이전트 도입 후 오히려 더 피곤해집니다."

PART 2

이 부하는 매일 기억을 잃습니다

기억 상실과 탐색 실패가 불러오는 3겹의 비용

어제의 세션과 오늘의 세션은 다른 사람입니다

어제 세션 (Session A)

"결제 실패 버그 원인을 분석해서 DB 인덱스 문제이고 `ORDER\_V2` 테이블은 건드리지 않기로 결정함."

오늘 새 세션 (Session B)

"깨끗한 빈 화면. 어제 무언가 논의했다는 사실 자체를 모름. 다시 `ORDER\_V2` 테이블을 고치자고 제안함."

 세션을 넘어서는 맥락이 팀 공용 기록으로 파일화되지 않으면 매번 원점으로 리셋

비용은 세 겹입니다

1. 재탐색

토큰·시간 낭비
(눈에 보이지만 가장 싼 비용)

2. 재현성 없음

그날 무엇을 읽었는가에 따라
결론이 매번 바뀜

3. 버렸던 대안의 재시도 ★

하지 않기로 한 대안을 잊어버려
잘못된 길을 다시 시도함 (가장 비쌈)

git이 기억하지 못하는 것

git이 기억하는 것

"무엇을 변경했는가" (코드 Diff, 커밋 메시지)

결과물의 코드 변경 내역

git이 기억하지 못하는 것

"무엇을 하지 않기로 결정했는가"

"왜 대안 B가 아닌 대안 A를 선택했는가"

기각된 선택지와 판단 맥락 (소실됨)

처음에는 괜찮다가 어느 순간 틀리기 시작합니다

프로젝트 초반 (파일 < 20개)

코드량이 적어 세션 시작 시 재탐색하는 것만으로 기억을 대체할 수 있음 (증상 없음)

프로젝트 성숙 (코드베이스 확장)

어디를 볼지 자체가 이전 세션의 지식 → 증상 전환: 느려지다가 틀리기 시작함

"틀림은 바로 눈에 보이지 않으며, 늦은 인프라 도입은 이미 증발한 결정을 복원하지 못합니다."

찾기는 구조적으로 실패할 수 있습니다

1. 어휘 불일치

내가 찾는 키워드 ≠ 기존 코드에 쓰인 키워드

2. 부재 증명 불가 ★

검색 0건 = 진짜 코드/규칙이 없는가?

3. 비문자열 연결

중요한 시스템간 의존성은 코드 문자열에 안 드러남

4. 부분 표본의 확신

파일 1~2개 훑고 전체 구조를 다 파악했다고 착각

"없습니다"는 결론이 아니라 가설입니다

 에이전트: "해당 함수를 검색한 결과 0건이므로 새로 작성하겠습니다."

→ 기존 유틸리티가 `scripts/legacy_helper.py`에 다른 이름으로 존재하는데도
조용히 중복 작성이 태어나는 결정적 오판 지점

 운영 규칙: 에이전트의 "없습니다"에 대해 **진입점** 지도/인계 노트를 사전에 줘어준다.

찾지 않아도 되게 만들어 줍니다

진입점 (Entry Point)

매 세션 접속 시 처음 읽어야 할 핵심 지도를 파일로 명시 (AGENTS.md / README)

상태 파일 (State File)

이전 세션이 어디까지 했고 무엇을 확인 못 했는지 인계 노트 제공

PART 3

신입이 한 명이 아닙니다 여럿이면 문제가 종류를 바꿉니다

혼자일 때와 팀일 때 생기는 파편화의 차이

아무의 머릿속에도 없는 결정이 생깁니다

혼자 일할 때

"내가 머릿속으로 기억하면 되지"가 어느 정도 통함. 임시 판단도 내 머리에 남아 있음.

여럿이 각자 에이전트로 일할 때

팀원 A와 에이전트 A1, 팀원 B와 에이전트 B1이 각자 다른 서랍에 **아무의 머릿속에도 없는 암묵적 결정**을 코드에 심어놓음.

중복은 충돌보다 늦게 발견됩니다

💣 충돌 (Conflict)

동일 파일 동일 줄을 건드림.

→ git merge 시 그 자리에서 에러로 드러남 (안전)

👻 중복 (Duplication)

각자 다른 파일/다른 함수명으로 같은 로직 구현.

→ 조용히 머지되고 한참 뒤 버그로 발견됨 (위험)

문제를 다 모았습니다

1. 기억 상실

세션이 닫히면 어제의 맥락 소실.
매 세션 처음부터 재탐색하는 비용 발생.

2. 탐색 오판

검색 0건을 '없다'고 오판.
기존 코드를 두고 조용한 중복 작성 태어남.

3. 협업 파편화

각자 에이전트로 암묵적 결정 생산.
충돌보다 위험한 조용한 중복 머지.

💡 이 3가지 문제 사슬을 끊는 결론 → 저장소 수준의 '워크플로우'

PART 4

워크플로우 — 여기서 이름을 붙입니다

사람의 성실성에 의존하던 지식을 저장소로 내려놓는 것

문제마다 붙일 장치가 다릅니다

1. 기억 상실 문제

→ 메모리 & 상태 인계 파일

지난 세션의 작업 위치와 인계 노트를 다음 세션 첫 줄에 즉시 복원합니다.

2. 규모·탐색 오판

→ Wiki & 진입점 인덱스

검색 없이 처음 읽어야 할 핵심 지도(AGENTS.md)를 주어 사각지대를 막습니다.

3. 중복 작업 문제

→ 작업 사전 공개 (백로그)

각자 돌리는 에이전트의 작업 범위를 사전에 공개해 중복 작업을 방지합니다.

4. 중구난방 산출물

→ 규칙 한 벌 (Rules)

사본 드리프트 없이 저장소 중앙의 동일한 규약 한 벌을 전원에 강제합니다.

사람이 읽는 문서와 AI가 읽는 문서는 다릅니다

사람용 문서 (Human Docs)

- **목적:** 맥락 이해, 의사결정 비하인드 파악, 팀 회고
- **특징:** 시각적 구조(헤더·요약), 친절한 서술, 가독성 중심
- **종류:** 기능 명세서, 시스템 아키텍처, 의사결정 기록(ADR)

AI용 문서 (Machine Context)

- **목적:** 환각 방지, 0.1초 단일 파싱, 조건 기계적 대조
- **특징:** 토큰 효율성, 명확한 정규식·경로, Key-Value 구조
- **종류:** `AGENTS.md`, `.agent/state.json`, 검사 스크립트

💡 둘을 분리하면 작성 공수는 절반으로 줄고, 에이전트의 참사 비율과 토큰 비용은 10배 절감됩니다.

오래된 지도는 없는 지도보다 나쁩니다

✖ 검색 실패: "못 찾았다"로 안전하게 끝남.

📅 Stale Wiki (오래된 문서): 틀린 지식을 에이전트가 아주 자신 있게 읽고 잘못된 코드를 짤.

"문서를 자동으로 생성하거나, 신선도를 검사하는 장치가 반드시 따라와야 합니다."

남기는 것만큼 버리는 것도 정합니다

1단계

세션 전량 기록 (Raw Log)

세션 내 모든 대화 및 코드 커밋 내역 수집

2단계

핵심 결정 추출 (Decision Log)

왜 대안 A를 선택하고 대안 B를 버렸는지 이유 추적

3단계

지식 승격 (Doc Elevation)

검증된 결정을 팀 위키/진입점 문서로 정제 상향

4단계

오래된 히스토리 폐기 (Purge)

승격 완료된 오래된 세션의 파편 로그 자동 청소

규칙은 한 벌만 씁니다

✕ 손 복사 방식

- 프로젝트마다 `.cursorrules`, `AGENTS.md`를 손 복사
- 시간이 지나며 사본 간 규약 드리프트 발생
- 테스트가 에러를 잡지 못해 오랫동안 방치됨

✓ 단일 원본 + 동적 생성

- 저장소 중앙에 단일 원본 규칙 한 벌만 유지
- 각 IDE/툴에 맞는 규약 파일은 빌드 시 자동 생성
- 사본 드리프트 없이 100% 동일한 컨벤션 보장

사례 A — 매뉴얼을 네 부 복사했더니

01. 이렇게 됐다

4부의 사본이 갈라짐

Task ID 정규식 규약이 4곳에 복사되었다가 사본 간 차이 발생

02. 왜 못 잡았나

검사 부재

서로 달라도 아무 테스트도 실패하지 않아 1년간 방치됨

03. 무엇을 붙였나

단일 출처화

원본 한 벌 설정 및 빌드 타임 자동 생성 스크립트 연결

04. 당신 팀에서는

사본 수색

팀 내 복사된 rules 파일들을 수색해 단일 출처로 통합

자연스러운 시스템화 = 다음 행동 안에 들어가는 것입니다

1. 에이전트가 쓴다

사람이 수동으로 쓰지 않고 세션 종료 시 에이전트가 인계 노트를 자동 작성

2. 커밋에 동봉한다

코드 변경 커밋에 인계 노트 파일도 단일 커밋으로 동봉하여 이력 관리

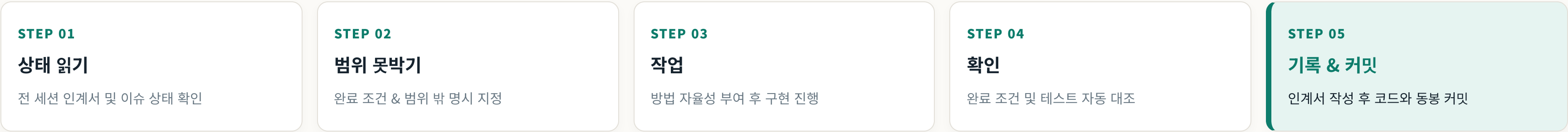
3. 안 하면 다음이 막힌다 ★

(예: 이전 세션 인계서가 없으면 오늘 에이전트가 '이전 상태 확인 불가' 경고 후 작업을 자동 중단)

4. 빠지면 알려준다

다음 세션 브리핑 첫 줄에 인계 파일 누락 사실을 즉시 알려 리더가 인지

하루 한 사이클



PART 5

그런데, 그 기록은 사실입니까

자동 기록의 허위 보고를 검사하는 최소 검증 장치

"썼다"와 "맞다"는 다릅니다

사례 B: 다른 서랍에 인계

읽는 쪽 reader만 새 경로로 바꾸고 쓰는 쪽 writer는 옛 자리에 둔 채 "인계서 갱신 완료" 보고서 제출.

사례 C: 미완료 작업의 '완료' 오분류

상태 미분류건이 fallback 처리로 Done 승격되어, 완료된 적 없는 3건이 완료 목록에 잘못 집계됨.

🔴 검사는 "썼는가"가 아니라 "쓴 내용이 실제 사실(State)과 맞는가"를 검증해야 합니다.

검사는 위험에 맞는 강도로 겁니다

1단계: 알림

다음 세션 브리핑 첫 줄에 뜨게 함.
커밋을 막지 않음.

2단계: 확인 요청

불일치 발견 시 사용자의 explicit 승인 후 진행.

3단계: 차단

치명적 규약 위반 시 작업 및 커밋 중단.

사람 신입과 AI 에이전트의 결정적 차이

사람 신입

- 경험이 쌓이고 온보딩은 한 번
- 막히면 질문을 던짐
- 신뢰가 쌓여 확인이 줄어들음

AI 에이전트

- 매 세션 기억 상실
- 되묻지 않고 짐작함
- 맥락/신뢰가 미누적됨

그래서 워크플로우란

- 온보딩 자동화
- 사각지대 문서화
- 확인을 장치 이관

좋은 리더는 자기 없이 굴러가게 만듭니다

지시는 문서로

모호한 말 대신 완료 조건과 범위 밖을 사전에 파일 한 문장으로 완벽하게 명시합니다.

맥락은 기록으로

세션별 주요 결정 이유와 거절했던 대안을 저장소 인계 노트로 차곡차곡 축적합니다.

확인은 장치로

리더의 주관적 검수 피로 대신 테스트 및 자동 검사 스크립트에 대조를 위임합니다.

CTA & 결론

지금 사람의 기억에만 맡겨둔 것은 무엇입니까

오늘 반나절이면 충분합니다:

- ① 진입점 파일 1개 (AGENTS.md / README)
- ② 기록을 남길 위치 1개 (상태/인계 파일: state / notes)
- ③ 최소 검사 스크립트 1개 (verify script)

참고 문서: [standard-ai-workflow docs](#)

감사합니다 (Q&A)